Tonic Characterization Literature: Self-Study Walkthrough

Analysis-driven rebuild of the literature deck's paper modules

Hongtao Zhang

April 13, 2026

This revision is for **offline learning**: each major paper now gets a **paper-first teaching module** with the paper's question, method or mechanism, evaluation setup, concrete findings, and only then the Tonic lesson.

How to read this deck

Use the same lens on every paper

1. What question is the paper trying to answer?
2. What mechanism or measurement method does it actually use?
3. What is the technically interesting trick, structure, or design choice?
4. What benchmark or comparison structure gives the numbers meaning?
5. What should Tonic measure differently because of that result?

Six buckets to keep stable

runtime / queueing, serialization / deserialization, copy / buffer lifecycle, compression / decompression, framing / transport glue, and wire / tail behavior

How this differs from the older deck

The older version synthesized the papers into Tonic lessons quickly. This one slows down and teaches what each paper is doing before using it.

What counts as a useful lesson

A paper matters here both because it helps Tonic and because it may teach a broader systems idea: a measurement strategy, a benchmark-construction trick, a hardware/software co-design, or a layout policy worth learning on its own.

What question should the literature answer?

The repo-specific question

1. serialization / deserialization
2. copies and buffer lifecycle
3. RPC stack processing and framing
4. queueing / scheduling / runtime policy
5. compression and host-side transforms
6. wire contribution and tail latency

Why bucketing matters

It prevents "RPC overhead" from collapsing payload work, stack work, queueing, and tail behavior into one average number.

What literature can answer

It can justify the decomposition vocabulary, show credible experiment design, and point to mechanisms worth isolating.

What literature cannot answer

It still cannot tell us which bucket dominates this repo's Tonic stack under each workload regime; only local measurement can do that.

Paper map: which paper helps with which bucket?

Bucket or design question	Strongest paper	What the reader should learn
queueing / stack tax / tails	Cloud-Scale RPC Characterization	How a production measurement paper separates application work from RPC tax and keeps queueing, stack work, and wire time visible.
serialization / deserialization	Hardware Accelerator for Protocol Buffers	Why protobuf behavior is rich enough to deserve its own benchmark, its own bucket, and its own accelerator story.
gRPC measurement method	TF-gRPC-Bench	How to derive workload-shaped microbenchmarks that separate serialized payload cost, iovec shape, and transport.
deployable acceleration	RPCAcc	Why acceleration wins only when serializer design, placement, and PCIe traversal are treated together.
copy / layout policy	Cornflakes	Why copy policy and serialization layout belong together, with a hybrid copy vs scatter-gather decision instead of universal zero-copy.
API-preserving transport replacement	RR-Compound	Why “keep the gRPC interface, replace the internal path” is a real design point, but only qualitatively grounded in this repo pass.

No single paper in this set gives an open tonic-like, stage-by-stage decomposition across serialization, copies, framing, scheduling, compression, and tail behavior.

Cloud-Scale RPC Characterization: the decomposition to remember

Why this paper anchors the deck

Paper type

A measurement paper: it does not introduce a new RPC mechanism, it introduces a production-scale decomposition for where RPC completion time is spent.

Core contribution

The paper forces a first-order split of **application processing** from **RPC latency tax**, then breaks the tax into queueing, RPC-stack, and wire terms.

Step 1 — split completion time

The paper separates application execution from **non-application RPC work**. The resulting tax includes queuing, stack, and network wire terms.

This prevents "just latency" summaries from hiding mechanism-level contributors.

Step 2 — open the RPC tax

Tax is decomposed into request/response queues, request/response RPC processing + network stack work, and request/response wire delay along both directions.

The measured model names the client/server queue and stack terms explicitly, not just "network delay."

Step 3 — keep tails explicit

The same component model is then used for heavy tails, nested RPC trees, and service-specific studies, so average tax and tail tax can diverge sharply.

Decomposition is a tail and workload-skew problem too.

Cloud-Scale RPC Characterization: how the study earns authority

Methodology visual

Monarch over 700 days

Production metrics are sampled every 30 minutes (700 days from Dec 2020 through Nov 2022), covering 10,000+ methods and service-level fleet behavior.

Long-window fleet trend baseline

Dapper trace decomposition

Per-RPC traces expose client/server/network components and nested RPC trees; the deep-dive trace pass covers one day with 722B sampled RPCs, and per-method tail analysis keeps only methods with at least 100 samples so P99 is well-defined.

Study scope from the paper text

The paper covers 10,000+ methods, 1B+ traces, 100s of clusters, and a 700-day production window spanning internal Google services built mostly on Stubby with some gRPC.

What the study can compare

Method and service distributions, cluster variation, request/response size distributions, call-tree shape (ancestors/descendants), latency components, and CPU-cycle tax are compared in one framework.

Why this matters

This is why the decomposition is believable: metrics, traces, and cycle profiles cross-check each other before the authors make method-level claims.

Request, response, and nesting in one model

GWP CPU profiling

Daily sampled CPU profiles add RPC-cycle decomposition to latency work so completion time and processing cost are viewed together.

Latency plus CPU-cycle tax

Cloud-Scale RPC Characterization: findings, limits, and Tonic lesson

Study context	Reported finding	Why it matters
fleet-scale growth trend	RPC throughput per CPU cycle rises by about 64% over 700 days (~30% annual growth), using metrics from nearly 23 months of observation.	RPC load is rising faster than CPU efficiency, so characterization has to handle both demand growth and changing mix.
method-level latency variation	90% of services have median latency >= 10.7 ms; 50% of methods have P99 >= 225 ms; 99.5% have P99 >= 1 ms; the slowest 5% have P1=166 ms and P99 >= 5 s.	Hyperscale RPC latency is heavily non-microsecond, so Tonic characterization should always include tail behavior and regime separation.
service/traffic concentration	Top-8 services already dominate volume patterns; storage-heavy services like Network Disk can drive many calls and bytes while being relatively cycle-light per RPC, and ML/F1 consume more cycles with far fewer calls.	This motivates service-specific optimization and why "average by method" is too coarse for planning optimization effort.
popularity versus total time	The top 100 lowest-latency methods are 40% of calls, but the slowest 1000 methods are only 1.1% of calls and still consume 89% of total RPC time.	You may need to target infrequent-but-expensive methods because they dominate time, not just frequency.
fleet-average tax versus high-overhead tails	Overall average tax = 2.0%; median method tax ratio = 8.6%; top 10% overhead methods have 38% median tax and P90 = 96%; network wire and RPC-processing-plus-network-stack together are major tail contributors.	A single low average does not imply low tail risk: tax dominance is concentrated and often tail-driven.
fleet-wide RPC cycle tax	RPC work consumes about 7.1% of all fleet CPU cycles; compression is 3.1%, networking 1.7%, and serialization 1.2% of total cycles.	The paper does not just decompose latency. It also shows why compression, networking, and serialization should remain separately visible in CPU-cost attribution.

The paper-text lesson is a **pattern of concentration**: average tax is modest, but service/method skew plus tail-heavy methods make the tax operationally critical.

Limitation

Internal Google RPC stacks (mostly Stubby/gRPC-like) and one hyperscaler context give strong directional evidence, but not direct implementation-level hook equivalence for Tonic.

Tonic lesson

Keep runtime, queueing, stack work, wire contribution, and tails visible by design. Averages and p99 are useful, but neither should be used alone for optimization decisions.

A Hardware Accelerator for Protocol Buffers: why protobuf gets its own bucket

What question this paper asks

Core claim

Protocol buffer serialization and deserialization are rich enough and costly enough to deserve dedicated study instead of being buried inside generic RPC cost.

Why this matters for the deck

The interesting systems lesson is not just "hardware wins." The paper first explains why protobuf is a serious workload class, then makes a non-obvious placement choice for the accelerator.

What the module must teach

protobuf behavior is diverse; `HyperProtoBench` exists because message structure and field mix matter; the accelerator is intentionally **near-core and instruction-dispatched**, not a NIC offload, because fleet data says much protobuf work is not RPC-bound

What not to over-claim

A protobuf accelerator result does not automatically explain runtime, framing, queueing, or buffer-movement cost in Tonic.

Best use in this repo

Use it to justify keeping prost encode/decode visible as a separate bucket and to justify workload-derived serialization benchmarks before optimization claims.

A Hardware Accelerator for Protocol Buffers: workflow and accelerator path

Methodology to mechanism

Step 1 — characterize protobuf usage

The paper profiles protobuf behavior at Google scale so the benchmark is derived from observed service-side usage rather than a toy message.

Profiling comes first

Step 2 — derive `HyperProtoBench`

That profiling becomes six synthetic benchmarks intended to reflect real workload diversity and field mix.

Benchmark design is part of the contribution

Evaluation setup

Six synthetic `HyperProtoBench` workloads, an RTL accelerator inside a RISC-V SoC, FireSim-based Linux evaluation, and comparison against BOOM-based and Xeon-based software contexts.

Why the chip design is interesting

Their fleet study argues against a simple NIC-placement story: only a minority of protobuf cycles are RPC-related, so the design favors low-latency instruction-level dispatch near the CPU instead of a PCIe-attached offload path.

Why the workflow matters to Tonic

This paper shows the profiling -> benchmark -> prototype chain that a serious accelerator argument should follow.

Step 3 — choose a near-core architecture

The accelerator is integrated into a Linux-capable RISC-V SoC, receives custom RoCC instructions directly from the BOOM core, and shares memory through the coherent TileLink path instead of sitting behind PCIe.

The architecture choice is itself a major lesson

Step 4 — reduce software bookkeeping overheads

The protobuf compiler generates one per-message-type Accelerator Descriptor Table (ADT), the design uses sparse hasbits for field presence, and the serializer walks fields in reverse order so sub-message lengths can be filled cleanly while staying wire-compatible.

Interesting microarchitecture, not just a benchmark score

A Hardware Accelerator for Protocol Buffers: results, limits, and Tonic lesson

Evaluation context	Reported result	Why it matters
benchmark basis	Six synthetic `HyperProtoBench` workloads are derived from Google service-side protobuf behavior.	The benchmark itself is evidence that protobuf behavior is diverse enough to characterize, not just accelerate.
BOOM-based baseline SoC	Average `6.2x`–`11.2x` speedup over the baseline SoC.	The accelerator is not a marginal optimization in the near-CPU SoC setting used by the paper.
Xeon-based server comparison	Average `3.8x` speedup over a Xeon-based server.	The result stays meaningful even when the comparison is not limited to the BOOM baseline.
prototype form	The prototype is a wire-compatible RTL accelerator integrated into a RISC-V SoC with custom-instruction dispatch, not a PCIe SmartNIC design.	That broader design choice is what makes the paper memorable: it links characterization, benchmark design, and a concrete near-core architecture while preserving the wire format.

Read these as **protobuf-path comparisons**, not as end-to-end gRPC transport numbers. The broader lesson is architectural too: this is a **near-core, instruction-dispatched** accelerator, very different from a PCIe- or NIC-only offload model.

Limitation

Tonic lesson

This is not a gRPC end-to-end paper and not a deployable NIC-attached design. It does not answer runtime, queueing, or HTTP/2 framing questions.

Keep prost encode/decode visible as a first-class bucket. Workload characterization should come before any serialization optimization story.

TF-gRPC-Bench: from TensorFlow traffic to benchmark classes

Why the paper exists

Problem

Full TensorFlow training runs are a noisy way to study the gRPC communication path if the real question is where communication cost comes from.

Paper type

A methodology paper: the main contribution is how it turns real traffic into controlled gRPC experiments.

Step 1 — observe TensorFlow-over-gRPC traffic

The starting point is traffic analysis from popular TensorFlow models, not arbitrary benchmark inputs.

Real workload structure first

Step 2 — derive three benchmark classes

The communication patterns become point-to-point latency, point-to-point bandwidth, and parameter-server throughput microbenchmarks.

Three ways to ask the question

Step 3 — keep serialization visible

Serialized and non-serialized modes are both used so transport-side behavior can be distinguished from serialization overhead.

Transport isolation is deliberate

Step 4 — expose payload shape as a knob

The benchmark does not treat payload construction as fixed: it lets users vary iovec count, buffer-size classes, and uniform/random/skewed payload generation so communication structure stays visible.

Workload generation is part of the mechanism

TF-gRPC-Bench: experiment matrix and controls

Controlled dimension	Values used	What it isolates
benchmark class	point-to-point latency, point-to-point bandwidth, parameter-server throughput	Which communication behavior matters for the workload: latency, bulk transfer, or update throughput.
serialization mode	serialized vs non-serialized	How much of the observed effect belongs to payload formatting rather than the underlying transport path.
payload / iovec shape	uniform / random / skewed iovec distributions, configurable buffer count and size	Whether message layout and buffer distribution change the measured transport behavior; this is one of the paper's best benchmark-design ideas.
transport	40G Ethernet, 10G Ethernet, IPoIB, RDMA	How the same communication pattern behaves when the underlying path changes but the workload shape stays matched.

This is why TF-gRPC-Bench matters so much to this repo: matched controls are part of the result, not just setup trivia. The broader lesson is benchmark craftsmanship: they model the parameter-server pattern and even use gRPC core C APIs in non-serialized mode to strip away serializer noise.

TF-gRPC-Bench: reported findings and what to copy into Tonic

Benchmark setting	Reported result	Why this comparison matters
64 KB serialized payloads	RDMA cuts point-to-point latency by about 40% versus 40G Ethernet and IPoIB in the reported setup.	With serialization still in the path, transport gains are visible but moderated by payload-formatting cost.
non-serialized skewed payloads on one cluster	RDMA cuts latency by about 59% versus Ethernet and 56% versus IPoIB.	Removing serialization overhead exposes a larger transport delta under the same skewed payload shape.
non-serialized skewed payloads on another cluster	RDMA cuts latency by about 78% versus 10G Ethernet and 69% versus IPoIB.	The magnitude changes with the cluster and link context, which is why the experiment matrix has to stay explicit.
parameter-server throughput	Throughput improves by roughly 4.1x versus 40G Ethernet, 3.43x versus IPoIB, and 5.9x versus 10G Ethernet in the reported settings.	The workload-shaped throughput case shows that the transport choice matters differently from the point-to-point latency cases.

The serialized vs non-serialized split is what makes the transport result interpretable: the bigger wins appear once payload-formatting cost is controlled separately. The suite also includes a bandwidth class; this deck emphasizes latency and parameter-server cases because the local notes preserve those comparisons most clearly, but the mechanism is the full three-class matrix.

Limitation

The traffic model is TensorFlow-specific and the transport context is 2018-era; this is a methodology model, not a direct Tonic headline source.

Tonic lesson

Build workload-shaped, matched microbenchmarks. Separate payload formatting from transport choice and keep payload shape explicit instead of treating it as background noise.

RPCAcc: deployment problem and bottleneck model

The problem the paper is solving

Naive offload is not enough

A PCIe-attached accelerator can lose much of its theoretical benefit if serializer behavior and placement are still movement-heavy.

Why the paper argues this matters

The repo's source trail records Google fleet motivation that RPC processing occupies about `7.1%` of CPU cycles.

Bottleneck model

serialization cost, PCIe traversal, receive-side placement, and repeated field updates all interact; none of them can be treated as a side detail

Why this belongs in the deck

It is the clearest modern paper in the repo set showing that deployable RPC acceleration is a constrained co-design problem, not a generic "put it on hardware" move.

What the module must teach

What PCIe breaks, what the three named techniques actually do, and why each reported number has to stay attached to its workload context.

RPCAcc: host-to-accelerator datapath and named techniques

Mechanism visual

Step 1 — memory-affinity collaborative serializer

RPCAcc adds a host pre-serialization phase that copies only CPU-resident fields into a contiguous DMA-safe buffer, can offload large copies to CPU memcpy engines, and leaves varint encoding to hardware so it does not pointer-chase nested CPU objects over PCIe.

Avoid slow pointer-chasing reads across PCIe

Step 2 — PCIe-attached on-NIC accelerator

Evaluation contexts

HyperProtoBench, end-to-end cloud workload experiments, and an image-compression latency case on the FPGA prototype path.

Why the datapath view matters

The paper is relevant to this repo because it ties host serialization, PCIe movement, accelerator work, and receive-side layout into one story.

What to remember

Acceleration helps only when traversal-aware design turns movement cost into something the accelerator can actually avoid; the memorable tricks are one-shot DMA write, CPU pre-serialization, and dynamic schema placement.

Programmable compute units and the RoCE-based path live in a deployable PCIe-attached design rather than an idealized on-die accelerator.

Deployment realism is the point

Step 3 — target-aware deserializer

The receive path uses a schema table plus a 4 KB temp buffer so fields can be batched in SRAM and flushed with one-shot DMA writes only when needed, instead of paying a PCIe transaction per field.

Avoid per-field PCIe writes on receive

Step 4 — automatic field updates

When kernels move between CPU and accelerator, RPCAcc updates schema placement metadata automatically so later

requests deserialize fields into the right memory without manual re-tagging.

Avoid stale placement causing extra traversals

RPCAcc: results, limits, and Tonic lesson

Evaluation context	Reported result	Why it matters
motivating fleet picture	RPC processing occupies about 7.1% of CPU cycles in the motivating fleet picture carried into the analysis artifact.	The CPU slice is large enough to motivate acceleration, but it does not by itself say how to accelerate safely.
HyperProtoBench	About 2.3x lower RPC-layer processing time than a comparable accelerator baseline.	The controlled benchmark result says the mechanism beats a more naive accelerator design, not just pure software.
end-to-end cloud workloads	Throughput improves by about 2.6x in the reported settings.	The benefit survives outside the benchmark harness, so the paper is not only a synthetic microbench story.
image-compression case	Average latency improves by about 2.6x, p99 by about 1.9x, and serialization time by about 4.3x geometric mean in the accessible analysis trail.	Latency, tail, and serialization improve by different amounts, which is exactly why one top-line multiplier is not enough.

RPCAcc is most useful when the results stay attached to **which workload and which metric** they come from; the paper does not collapse them into one universal win. It is also a strong lesson in **PCIe-aware co-design**: treat transaction rate, pointer chasing, and placement as first-class constraints.

Limitation

Tonic lesson

This is a 2024 arXiv FPGA prototype, not a deployed tonic-specific production stack. The paper does not answer Rust async scheduling or Tonic framing questions directly.

Treat serialization, placement, and interconnect traversal as a coupled design space. A single throughput headline is not enough evidence for later offload claims.

Cornflakes: hybrid copy vs scatter-gather mechanism

Why this paper is adjacent but useful

Problem

Serialization overhead is entangled with packet assembly and extra copies in microsecond-scale networking.

Core point

Pure zero-copy is not always optimal. The paper argues for a **hybrid** choice between ordinary copies and scatter-gather transmission.

Step 1 — schema-driven hybrid pointers

Cornflakes-generated objects represent string/bytes fields as `CFPtr`, a hybrid pointer that is either copied data or a reference-counted DMA-safe buffer (`RcBuf`).

Layout policy is part of serialization

Step 2 — networking runtime integration

The stack can either flatten and copy or hand discontiguous chunks to the NIC through scatter-gather, and its combined serialize-and-send API avoids first materializing an intermediate scatter-gather array.

Copy path vs scatter-gather path

Step 3 — choose the cheaper path

The hybrid decision uses a per-field size threshold: on their hardware, zero-copy starts paying off around `512 B`, while smaller fields are cheaper to copy once metadata/cache-miss overhead is included.

Zero-copy is regime-dependent

Cornflakes: evidence, limits, and implication for Tonic

Evaluation path	Accessible evidence	How to read it
custom KV store on a Twitter cache trace	`15.4%` higher throughput.	The hybrid copy / scatter-gather policy can pay off in a realistic service-like path.
inside Redis	`8.8%` higher throughput relative to Redis serialization.	The paper is not limited to a custom toy runtime; layout policy matters even in an established software stack.
precursor-path intuition, not primary SOSP evidence	About `9.15 Gbps` highest throughput while staying under `15 µs` tail latency.	The repo analysis explicitly treats some of the strongest convenient numbers as coming from the precursor path, not a full direct SOSP evaluation read.
mechanism intuition	Their mechanism combines DMA-safe `RcBufs`, `CFPtr` hybrid pointers, a `512 B` zero-copy threshold, and serialize-and-send to remove intermediate array construction.	This is why the paper belongs in the copy / layout bucket rather than as a transport-replacement citation: the cleverness is in policy and API co-design.

This module stays concrete but explicitly **adjacent**: the custom KV and Redis rows are the main paper-grounded evidence used here, while the precursor-path row is only mechanism intuition. The broader learning value is the **hybrid API design**, not just the throughput delta.

Limitation

This is not a gRPC paper, and the strongest accessible numeric details are partly stronger in the precursor trail than in a directly recovered full SOSP body.

Tonic lesson

Keep copy policy and serialization layout visible. “Zero-copy” is not a universal answer; it is a workload- and bookkeeping-dependent decision.

RR-Compound: explicit low-confidence qualitative design-space note

What is safely known

1. compatibility-first gRPC-over-RDMA framing
2. RDMA-enabled internal fast path from the artifact-repo view
3. runtime-tunable transport knobs exist in the artifact

Artifact-level mechanism hints, not paper results

RDMA disabled by default, one polling thread, `500` µs busy-poll timeout, and `4096` KB ring buffer per connection suggest a

What is missing

This repo pass does not recover enough paper text to teach the evaluation deeply or to trust a numeric headline the way we can for the other modules.

How to use this citation

Use RR-Compound only as design-space context: API-preserving transport replacement is plausible, but the present evidence is qualitative.

Why this slide is explicit

The rebuild analysis says this module should stay low-confidence until a stronger paper-ingestion pass is done. This slide teaches the design goal, not the evaluation.

compatibility-first fast path with explicit runtime tuning rather than a magical transparent replacement.

How the literature buckets map onto the Tonic byte path

Outbound path: where the bytes are created

encode

prost encode of the outbound message

protobuf stays visible because of the MICRO 2021 paper

buffer reserve / grow

reserve space and absorb growth / reallocation behavior

copy and layout policy stay visible because of Cornflakes and TF-gRPC-Bench

Inbound path: where the bytes are interpreted

body accumulation

collect the inbound body bytes before decode

body accumulation belongs to copy / buffer lifecycle, not to protobuf alone

frame parse

read compression flag and frame length

framing and transport glue are their own bucket

compression

optional software compression before the frame is emitted

compression is a separate CPU-tax bucket, not just "wire optimization"

frame-header write + body handoff

write the gRPC frame header and hand the body toward transport

stack and framing work stay visible because of the cloud-scale RPC decomposition

decompression

optional software decompression on the receive side

compression trade-offs must be interpreted on both throughput and tail axes

prost decode + application handoff

decode the protobuf message and return to application code

this is the inbound mirror of the serialization bucket

`docs/report/architecture/003.tonic_interception_points.md` is the grounding artifact here: interceptors are metadata-only, so payload-path analysis belongs at the codec/body boundary instead.

Repo evidence already shows multiple regimes

Regime	Repo evidence from report 009	Dominant bucket	Characterization implication
256 B low concurrency, compress=off	single-thread beats multi-thread at c=1 (27.51k vs 20.97k rps); 2.1 IPC and 54.1% frontend-bound	runtime / control	Tiny unary RPCs should be read as runtime-sensitive before claiming payload offload value.
4 KiB medium concurrency, compress=off	multi-thread wins at c=128 (65.79k vs 40.59k rps); 41.3% backend-bound; memmove 10.51%, malloc 8.67%	copy + allocation	Buffer lifecycle has to be measured directly; one throughput number hides where the time is going.
1 MiB large, uncompressed	very large payloads stop benefiting from extra runtime parallelism (1063.55 vs 961.28 rps at c=512); 0.3 IPC; memmove 71.37% self time	movement-dominated	This is the clearest copy-path characterization lane, but it still needs software controls before any hardware conclusion.
64 KiB structured, compress=on	throughput drops (6.65k vs 12.45k rps) while p99 improves (30.00 ms vs 100.30 ms); compress_inner 61.77% self time	compression	Compression has to be interpreted on both throughput and tail latency axes; it is not globally good or bad.

Runtime crossover summary from 009: tiny structured/off RPCs favor single-thread at low concurrency, medium structured/off RPCs favor multi-thread, and very large payloads fall back to movement-dominated behavior.

Characterization rules derived from literature plus profiling

Characterize by bucket

Every result should resolve into runtime, serialization, copy/buffer lifecycle, compression, or framing/transport work rather than one throughput headline.

Characterize by regime

Matched size, concurrency, runtime, and payload kind matter because the dominant bucket changes with regime.

Use controls before claiming offload

If pooled buffers or copy minimization collapse an apparent hotspot, that hotspot is not yet strong evidence for later DSA or IAX work.

Separate throughput from tail latency

Compression already gives cases where throughput falls while p99 improves, so one metric cannot stand in for the whole characterization.

Concrete next-step measurement pass for Tonic

> The next pass is a codec/body-path refinement with stage timers and software controls — not premature accelerator integration.

Timers and hooks to add

1. encode / decode
2. compress / decompress
3. buffer reserve / grow
4. body accumulation / handoff
5. frame write / parse where visible

Controls to run

Instrumentation-off

Measure whether the timers themselves distort the path before using them for attribution.

Pooled-buffer

Reduce allocation and growth noise while keeping semantics unchanged.

These hooks follow the real byte path from Slide 20 rather than metadata-only interception points.

Copy-minimized

Remove avoidable copies to learn how much of the hot path is policy rather than irreducible work.

Runtime and placement controls

Re-run matched points under single vs multi runtime and controlled client/server placement.

Final takeaway

The literature gives the decomposition vocabulary and the experimental caution, but the answer still has to come from a Tonic-specific measurement campaign.

What is established

The six-bucket view is well motivated, and report `009` already shows that Tonic moves across multiple bottleneck regimes.

What remains missing

No external paper replaces a stage-by-stage, stack-preserving Tonic decomposition with controls and tail interpretation.

What comes next

Use codec/body-boundary timers, matched regimes, and software controls to learn which buckets still justify later accelerator follow-up.